

DDA5001 Machine Learning

Transformer (Part II)

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Information about Final

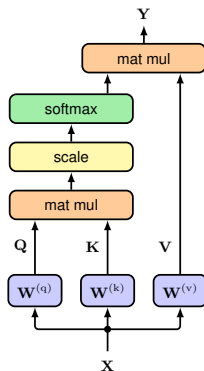
- ▶ Time: May 9 (Saturday), 9:00am - 11:00am (2 hours).
- ▶ A closed-book, closed-notes examination. No cheat sheet (this leads to an easier exam). No other things will be allowed, especially calculators.
- ▶ A sample exam will be available shortly to aid you in your review. Note that do not go overfitting, you need a little generalization. Hence, you also need to go through our lecture slides and Homework questions.

Recap: Self-Attention

- **Attention** is to consider the connections between different tokens in a sentence.
- We use self-attention to capture the semantic relationship:

$$Y = \text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Illustration:



Recap: Multi-Head Attention, Attention Masks

Multi-head attention: To capture different semantic structure by different attentions:

$$Y(X) = [H_1, \dots, H_H] W^{(o)}.$$

Each head H_i is generated by attention.

Attention masks:

- We need attention masks to block attention to **padding tokens** and **future tokens** (for **causality**):

$$Y = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M^{\text{causal}} + M^{\text{pad}} \right) V,$$

where

$$M_{ij} = \begin{cases} 0, & \text{allowed} \\ -\infty, & \text{blocked} \end{cases}$$

Agenda

Transformer

▪

Transformer Layer

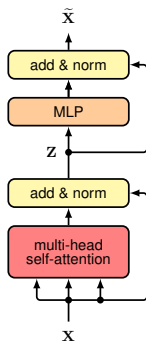
Transformer Layer

- ▶ We now need to form a **transformer layer**.
- ▶ We want to add some **residual connection** to improve efficiency.
- ▶ We want to add **layer normalization (LayerNorm)** for efficiency as well.
- ▶ We also want to add several MLP layer (feed-forward layer) to have more flexibility.

Transformer Layer

- ▶ We now need to form a **transformer layer**.
- ▶ We want to add some **residual connection** to improve efficiency.
- ▶ We want to add **layer normalization (LayerNorm)** for efficiency as well.
- ▶ We also want to add several MLP layer (feed-forward layer) to have more flexibility.

Overall, a transformer layer has the form:



- ▶ The mathematical modeling is:

$$Z = \text{Norm}(Y(X) + X).$$

$$\tilde{X} = \text{Norm}(\text{MLP}(Z) + Z).$$

- ▶ The dimension of the output $\tilde{X}$ need to be the same as that of the input X .

Norm Method I: Layer Normalization (LayerNorm)

Consider $\mathbf{x} = [x_1, \dots, x_d]$ as **one** data sample, like one row of $\mathbf{X}$ (i.e., one token). **LayerNorm** does the following:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}, \quad \text{where} \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

- The above process is to make the data zero mean and unit variance.

Norm Method I: Layer Normalization (LayerNorm)

Consider $\mathbf{x} = [x_1, \dots, x_d]$ as **one** data sample, like one row of $\mathbf{X}$ (i.e., one token). **LayerNorm** does the following:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}, \quad \text{where} \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

- The above process is to make the data zero mean and unit variance.

LayerNorm adds two additional **trainable** parameter vectors

$\boldsymbol{\gamma} = [\gamma_1, \dots, \gamma_d]$ and $\boldsymbol{\beta} = [\beta_1, \dots, \beta_d]$.

For each dimension $i = 1, \dots, d$, we then scale (γ_i) and shift (β_i) $\hat{x}_i$ to obtain:

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

Finally, $\mathbf{y} = [y_1, \dots, y_d]$ will be the layer-normalized data sample.

- LayerNorm can be regarded as giving a new **learned** mean and variance for each data.
- LayerNorm is important to improving stability of the training process.

Norm Method II: RMSNorm

Another popular norm is **Root Mean Square Normalization (RMSNorm)**.

- First compute the root mean square (**RMS**)

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}.$$

where ε is a small number for numerical stability.

- Then, apply **normalization** using RMS

$$\hat{x}_i = \frac{x_i}{\text{RMS}(\mathbf{x})}, \quad y_i = \gamma_i \hat{x}_i.$$

γ_i is the trainable scaling factor.

Norm Method II: RMSNorm

Another popular norm is **Root Mean Square Normalization (RMSNorm)**.

- First compute the root mean square (**RMS**)

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}.$$

where ε is a small number for numerical stability.

- Then, apply **normalization** using RMS

$$\hat{x}_i = \frac{x_i}{\text{RMS}(\mathbf{x})}, \quad y_i = \gamma_i \hat{x}_i.$$

γ_i is the trainable scaling factor.

Comparison: LayerNorm both scale and shift, while **RMSNorm only scales**.

- Older / classic LLMs (GPT-2, GPT-3) uses LayerNorm.
- More recent very large LLMs uses RMSNorm.

Post-Norm and Pre-Norm

The method of applying LayerNorm outlined above is called **Post-Norm**:

$$\begin{aligned}Z &= \text{Norm}(\mathbf{Y}(\mathbf{X}) + \mathbf{X}). \\ \widetilde{\mathbf{X}} &= \text{Norm}(\text{MLP}(\mathbf{Z}) + \mathbf{Z}).\end{aligned}$$

- ▶ It post-normalizes output after residual connection (i.e., addition).
- ▶ This is the Norm used in the “Attention is All You Need” paper.

However, modern LLMs mainly use **pre-norm** which **pre-normalizes the input**:

$$\begin{aligned}\mathbf{Z} &= \mathbf{Y}(\text{Norm}(\mathbf{X})) + \mathbf{X}. \\ \widetilde{\mathbf{X}} &= \text{MLP}(\text{Norm}(\mathbf{Z})) + \mathbf{Z}.\end{aligned}$$

As it is found that pre-norm provides a more stable training process.

Complexity of One Transformer Layer

Consider post-norm transformer layer:

$$\mathbf{Z} = \text{Norm}(\mathbf{Y}(\mathbf{X}) + \mathbf{X}), \quad \widetilde{\mathbf{X}} = \text{Norm}(\text{MLP}(\mathbf{Z}) + \mathbf{Z}).$$

Notations. n : sequence length, d : model dimension, d_{MLP} : hidden size of MLP (often $d_{\text{MLP}} \approx 4d$).

- ▶ As we studied, the **attention module** has computation complexity $O(n^2d)$ and memory complexity $O(n^2)$.

In **MLP module** (the MLP is often two-layer):

- ▶ Computation complexity:

$$O(ndd_{\text{MLP}}) \quad (\text{two linear layers, with elementwise nonlinearity in between})$$

- ▶ Memory for activation values:

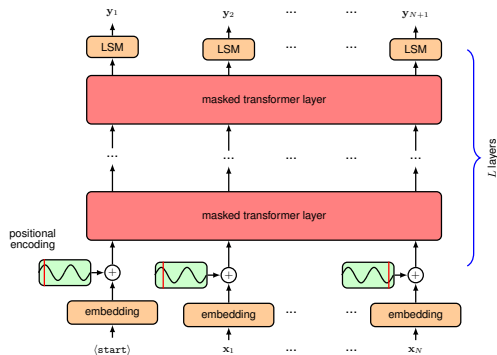
$$\mathcal{O}(nd_{\text{MLP}}) \quad (\text{need to store } d_{\text{MLP}} \text{ hidden activations in backpropagation})$$

So the overall order of complexity of the full Transformer layer is **quadratic in n** .

▪

Transformer Architecture

Decoder Transformer Architecture



- ▶ This is **decoder transformer**, which is the architecture of GPT, Llama, and many other popular language models.
- ▶ Transformer is to stack transformer layers multiple times (just like one hidden layer to multiple), and add some other modules.

Tokenization

Goal: Map raw text to a sequence of discrete **tokens**

“It is raining.” $\longrightarrow (t_1, t_2, \dots, t_n)$.

- ▶ Is token in the unit of a character or a whole word? None of them.
 - ▶ Characters: very long sequences $\Rightarrow$ inefficient.
 - ▶ Words: huge **vocabulary**, many rare/unknown words.

Tokenization

Goal: Map raw text to a sequence of discrete **tokens**

“It is raining.” $\longrightarrow (t_1, t_2, \dots, t_n)$.

- ▶ Is token in the unit of a character or a whole word? None of them.
 - ▶ Characters: very long sequences $\Rightarrow$ inefficient.
 - ▶ Words: huge **vocabulary**, many rare/unknown words.
- ▶ Modern LLMs use **subword tokenization**:
 - ▶ Methods: BPE (GPT2), SentencePiece (Qwen3), WordPiece, etc.
 - ▶ Common words are single tokens; rare words are split:

“unbelievable” $\rightarrow$ ‘‘un’’, ‘‘believ’’, ‘‘able’’

“raining” $\rightarrow$ “rain”, “ing”

- ▶ Each token is represented internally by an **integer id**

$$t_i \in \{1, \dots, V\},$$

where V is the **vocabulary size** (e.g., $V \approx 30\text{k} \sim 100\text{k}$).

Embedding Layer

- ▶ The model cannot work directly with token ids t_i ; it needs vectors.
- ▶ **Embedding matrix shared** by all embedding positions:

$$\mathbf{E} \in \mathbb{R}^{V \times d}, \quad \text{row } \mathbf{E}_{t_i:} = \mathbf{x}_i^\top \in \mathbb{R}^{1 \times d}.$$

Each token id t_i is mapped to an embedding vector $\mathbf{x}_i$:

$$t_i \longrightarrow \mathbf{x}_i = \mathbf{E}^\top \mathbf{e}_{t_i},$$

where $\mathbf{e}_{t_i}$ is the one-hot vector of t_i , i.e., taking out the corresponding row of $\mathbf{E}$.

Embedding Layer

- ▶ The model cannot work directly with token ids t_i ; it needs vectors.
- ▶ **Embedding matrix shared** by all embedding positions:

$$\mathbf{E} \in \mathbb{R}^{V \times d}, \quad \text{row } \mathbf{E}_{t_i:} = \mathbf{x}_i^\top \in \mathbb{R}^{1 \times d}.$$

Each token id t_i is mapped to an embedding vector $\mathbf{x}_i$:

$$t_i \longrightarrow \mathbf{x}_i = \mathbf{E}^\top \mathbf{e}_{t_i},$$

where $\mathbf{e}_{t_i}$ is the one-hot vector of t_i , i.e., taking out the corresponding row of $\mathbf{E}$.

- ▶ Embedding matrix $\mathbf{E}$ is **learned**, rather than designed.
- ▶ After adding positional encodings (represent order of words in your sentence), the sequence

$$\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$$

is fed into the stack of masked transformer layers shown earlier.

The Last Layer: Logistic Regression

- ▶ In the last **LSM** layer, it means linear transformation + softmax, i.e., **multi-class logistic regression** on the final learned feature $\mathbf{Z} \in \mathbb{R}^{(n+1) \times d}$, the added token is <start>.
- ▶ The number of classes is V , i.e., the **vocabulary size**. Using classification to **predict** the next token in the sequence, by choosing a token from the vocabulary.
- ▶ The classification is done using logistic regression with **lm_head** $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{d \times V}$, i.e., linear classification weight matrix.

Mathematically,

$$\mathbf{P} = [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_{n+1}]^\top = \text{softmax}(\mathbf{Z}\mathbf{W}_{\text{lm}}) = \text{softmax}(\mathbf{L}).$$

$\mathbf{P}$ is an $(n+1) \times V$ matrix. Every row corresponds to a distribution $\mathbf{p}_i$.

The Last Layer: Logistic Regression

- ▶ $L = ZW_{\text{lm}} = [l_1, \dots, l_{n+1}]^T \in \mathbb{R}^{(n+1) \times V}$ are called **logits** corresponding to each input token x_i .
- ▶ $p_i \in \mathbb{R}^V$ is the predicted probability distribution over the vocabulary, for **predicting the next token i** , as we have **shifted** the position by adding the starting token <start>.
- ▶ The groundtruth **label** for p_i is the real i^{th} token in the sequence/sentence: x_i .
- ▶ To summarize, the **learned feature vector for x_{i-1}** (i.e., the i^{th} row of Z) captures information of the sentence up to token x_{i-1} . We use this vector to obtain p_i and then use it to **predict the token x_i** (i.e., the label): **Next-Token Prediction**.

Interpretation: Transformer uses the new attention mechanism. Its overall idea is similar to other neural networks. Trying to extract useful features Z for linear classification.

Tied Embedding and LM Head

Tied parameters in LLMs:

- ▶ In decoder-only LMs, the **same** matrix (or its transpose) is often used for the input embedding and for the LM head matrix $\mathbf{W}_{\text{lm}}$ in the final logistic regression layer.

$$\mathbf{W}_{\text{lm}} = \mathbf{E} \quad (\text{or } \mathbf{W}_{\text{lm}} = \mathbf{E}^{\top}, \text{ depending on convention}).$$

- ▶ This is called Tied LM head.
- ▶ it reduces parameters by $V \times d$. This is often a very large matrix.

Tied Embedding and LM Head

Tied parameters in LLMs:

- ▶ In decoder-only LMs, the **same** matrix (or its transpose) is often used for the input embedding and for the LM head matrix $\mathbf{W}_{lm}$ in the final logistic regression layer.

$$\mathbf{W}_{lm} = \mathbf{E} \quad (\text{or } \mathbf{W}_{lm} = \mathbf{E}^\top, \text{ depending on convention}).$$

- ▶ This is called Tied LM head.
- ▶ it reduces parameters by $V \times d$. This is often a very large matrix.

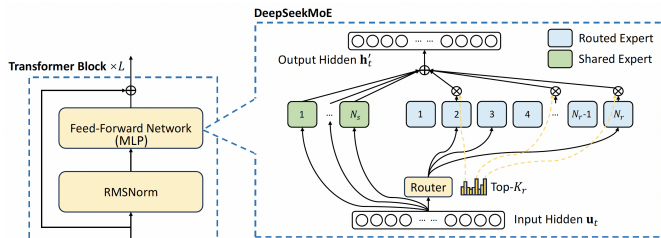
This is reasonable as we use the same embedding to transfer tokens to embedding vectors, and then use the same matrix to transfer embedding (i.e., final hidden state) back to tokens in the vocabulary.

In other words, given the model's final hidden state, we predict the next token as the one whose embedding is most similar (via dot product) to the final hidden state.

Mixture of Experts (MoE)

Mixture-of-Experts (MoE) Feed-Forward Layer

- ▶ MoE is to expand the **MLP / feed-forward network (FFN)**, not attention.
- ▶ In standard Transformers, each token goes through the **same** MLP layer.
- ▶ In an MoE MLP layer, we have multiple MLPs (**experts**) and a **router** that chooses a small subset of experts for each token.
- ▶ MoE is a simple architecture scaling approach that can expand the representation power of transformer (said by OpenAI people).



Mathematical Definition of MoE

- ▶ Let $\mathbf{u}_t \in \mathbb{R}^d$ be the hidden state / activation value of token $\mathbf{x}_t$ entering an MoE MLP. The router produces a sparse weighting over N_r experts:

$$\mathbf{g}_t = \text{Top-}K_r(\text{softmax}(\mathbf{R}\mathbf{u}_t)) \in \mathbb{R}^{N_r},$$

where $\mathbf{R} \in \mathbb{R}^{N_r \times d}$ is the router matrix and K_r is the number of activated experts per token (e.g., $K_r = 2$).

- ▶ Only the top- K_r entries of $\mathbf{g}_t$ are kept, others are 0, i.e., only the **top- K_r** routed experts are chosen.

Mathematical Definition of MoE

- ▶ Let $\mathbf{u}_t \in \mathbb{R}^d$ be the hidden state / activation value of token $\mathbf{x}_t$ entering an MoE MLP. The router produces a sparse weighting over N_r experts:

$$\mathbf{g}_t = \text{Top-}K_r(\text{softmax}(\mathbf{R}\mathbf{u}_t)) \in \mathbb{R}^{N_r},$$

where $\mathbf{R} \in \mathbb{R}^{N_r \times d}$ is the router matrix and K_r is the number of activated experts per token (e.g., $K_r = 2$).

- ▶ Only the top- K_r entries of $\mathbf{g}_t$ are kept, others are 0, i.e., only the **top- K_r** routed experts are chosen.

Overall, the MoE layer per token is mathematically defined as

$$\mathbf{h}'_t = \underbrace{\mathbf{u}_t}_{\text{skip connection}} + \underbrace{\sum_{i=1}^{N_s} \text{MLP}_i^{(s)}(\mathbf{u}_t)}_{\text{shared MLP}} + \underbrace{\sum_{i=1}^{N_r} \mathbf{g}_t[i] \cdot \text{MLP}_i^{(r)}(\mathbf{u}_t)}_{\text{routed expert}}.$$

↪ Next lecture: Large language models (LLMs).